

CENG4265: SENIOR PROJECT – FINAL REPORT

LIDAR SCOUT ROVER

by

Nicholas Boggs, Zabdiel Montano, Andrew Patino, Randy Rodriguez

Advisor: Dr. Luong Nguyen

Computer Engineering Program

Engineering Department

University of Houston-Clear Lake

April 24, 2026

ABSTRACT

LIDAR SCOUT ROVER

Nicholas Boggs, Zabdiel Montano, Andrew Patino, Randy Rodriguez

University of Houston-Clear Lake, 2026

Advisor: Dr. Luong Nguyen, PhD

The LiDAR-based Scout Rover project was developed to map spaces, navigate obstacle awareness, monitor gas readings, and display live sensor feedback within a ROS 2 Jazzy system. The final platform used a distributed architecture in which an Arduino Mega handled low-level motor and gas-sensor functions, a Raspberry Pi 5 published LiDAR, camera, and bridge topics, and a workstation ran Point-LIO, Nav2, RViz2, and the main visualization tools. During testing, the rover was able to generate LiDAR maps, publish gas data, stream camera output, and execute navigation experiments with both a front-focused obstacle pipeline and a later 360-degree costmap pipeline. The results show that a relatively compact multi-computer rover can support practical sensing and semi-autonomous navigation, while also highlighting future work in TF stability, software tuning, and demonstration reliability.

TABLE OF CONTENTS

List of Tables	vi
List of Figures	vii
CHAPTER 1: INTRODUCTION & BACKGROUND.....	1
1.1 Project Area:	1
1.2 Context of the Project:	1
1.3 Why Project/Problem is Worth Doing/Solving:	3
1.4 Theory:.....	3
CHAPTER 2: DESIGN AND METHODOLOGY	6
2.1 Project Statement:	6
2.2 Project:	8
2.3 Method:	9
2.4 Deliverables:	9
2.5 Requirements Gathering – Specifications:.....	10
2.6 Feature Analysis Based on our User Requirements.....	11
2.7 System Architecture and Components:.....	12
2.8 Mechanical Integration and 3D-Printed Parts:.....	14
CHAPTER 3: TECHNICAL INVESTIGATIONS & EXPERIMENTS.....	15

3.1 Evaluation & Selection of Tools (Software, Hardware, Platform):	15
3.2 Development of Prototype:	16
3.3 Testing & Debugging:.....	18
3.4 Usability Tests:	19
3.5 Results / Findings:.....	20
CHAPTER 4: PROJECT MANAGEMENT	23
4.1 Project Management:	23
CHAPTER 5: CONCLUSION AND FUTURE WORK.....	24
5.1 Conclusion and Future Work:.....	24
BIBLIOGRAPHY	26

LIST OF TABLES

Table 2.1 Rover Budget (Listed Components)	12
--	----

LIST OF FIGURES

Figure 2.4 3D-Printed Skid Plate in Fusion360	14
Figure 2.5 3D-Printed Front Plate in Fusion360.....	14
Figure 2.6 Point-LIO & NAV 2 System Flowchart.....	14
Figure 2.7 Electrical System Overview	15
Figure 3.2 NAV 2(Navigation 2) V1	18
Figure 3.2 NAV 2(Navigation 2) V2 (NAV 2 Goal Start)	18
Figure 3.5 NAV 2(Navigation 2) V2 (NAV 2 Goal Reached)	22

CHAPTER 1:

INTRODUCTION & BACKGROUND

1.1 Project Area:

The LiDAR Scout Rover falls within the area of mobile robotics, focusing primarily on autonomous navigation and environmental mapping. This project combines concepts from embedded systems, sensing technologies, and real-time data processing to design a rover capable of perceiving and interpreting its surroundings. By integrating a LiDAR sensor with onboard computation, the rover can generate spatial maps, detect obstacles, and plan movement paths without relying on external guidance. These capabilities are essential in modern robotic applications such as search and rescue operations, automated inspection, and exploration in hazardous or inaccessible environments.

The project also touches on areas like system integration and mechatronics, since the rover must coordinate mechanical components (motor and chassis), electronic hardware (motor drivers, microcontrollers and power systems), and software frameworks for control. The software side is especially important because it involves processing point cloud data, implementing SLAM (Simultaneous Localization and Mapping), and managing communication between different modules. Overall, the LiDAR Scout Rover combines the usage of robotics engineering and computer science, making it a practical example of how sensing and autonomy can come together in real-world robotics systems.

1.2 Context of the Project:

Our LiDAR Scout Rover was developed as part of a broader effort to explore practical applications of autonomous robotics within an academic setting. As sensing and automation technologies continue to advance, industries such as environmental monitoring and disaster response increasingly rely on robots that can safely navigate complex environments without human intervention. This project aims to mirror those real-world challenges by designing a small-scale rover capable of generating spatial awareness using LiDAR, a sensor commonly used in professional robotics for high-precision mapping.

In the context of this course, the project serves as a hands-on introduction to integrating hardware and software systems while applying concepts learned throughout the semester. It requires students to work across multiple disciplines such as mechanical design, electronics, embedded programming, and data analysis to build a complete functioning platform. The project also helps simulate real engineering workflows, such as selecting components based on constraints, troubleshooting system interactions, and validating performance through testing. By situating the rover in a realistic engineering scenario, the project highlights the importance of autonomous sensing systems and prepares students for more advanced work in robotics, automation, and intelligent systems. In addition to software development and sensor integration, the project required mechanical design and hardware safety to make the rover usable as a physical mobile platform. Several components, including the LiDAR, Raspberry Pi, webcam, gas sensors, batteries, buck converters, and wiring, had to be mounted securely on a compact tracked chassis. Because the rover was intended for repeated testing and demonstration, the physical layout needed to support stability, accessibility, and reliable power routing.

1.3 Why Project/Problem is Worth Doing/Solving:

The LiDAR Scout Rover project is worth solving because it addresses the growing need for reliable autonomous systems capable of navigating and understanding real-world environments. As robots become more common in fields like disaster assessment, environmental surveying, warehouse logistics, the ability to accurately map surroundings and avoid obstacles is essential. Developing a rover that can perform these tasks on its own provides valuable insight into the same challenges engineers face in larger-scale autonomous vehicles and larger robots.

From an educational standpoint, this project is also important because it pushes students to move beyond theoretical knowledge and apply engineering concepts in a realistic, hands-on way. It encourages problem solving across multiple disciplines such as software, hardware, sensing, and systems integration which mirrors the complexity of actual robotics projects. By working through issues like sensor calibration, power management, mechanical stability, and data interpretation, students gain practical experience that classroom exercises cannot replicate. The LiDAR Scout Rover represents a meaningful learning opportunity and demonstrates how autonomous sensing and navigation technologies can be designed, tested, and improved in a controlled academic environment.

1.4 Theory:

The LiDAR Scout Rover relies on several core principles in robotics and sensing to interpret its surroundings and generate usable environmental information. At the center of

the system is Light Detection and Ranging, or LiDAR, which measures distance by emitting rapid pulses of light and calculating how long they take to return after striking objects in the environment. When these measurements are combined across a full scan, the sensor produces a point cloud, which is a collection of three-dimensional coordinates that represent surfaces, walls, and obstacles around the rover. Point clouds form the basis for mapping and allow the system to understand the shape and layout of nearby areas.

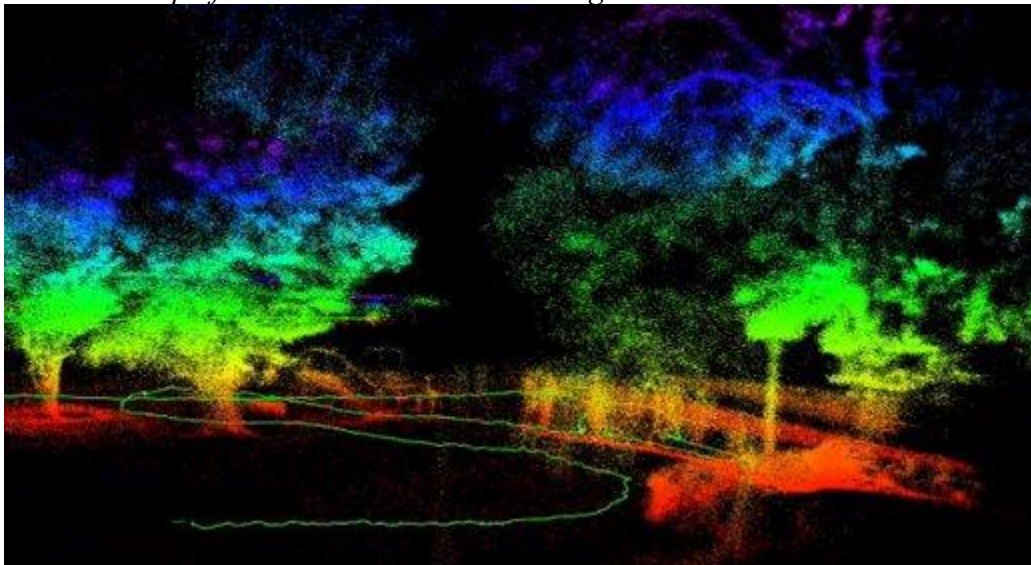
Unitree ROS 2 driver, which publishes a point cloud topic for the scans and an IMU topic if enabled. Two static transforms define the small kinematic chain from `base_link` to the IMU and from the IMU to the LiDAR so that all data has consistent frames. The laptop discovers those topics over the network, launches Point-LIO (with a `ros2 launch` command referencing our parameters file and the point cloud/IMU topic names), and displays results in RViz2. Within RViz2 we set the “Fixed Frame” to the mapping frame used by Point-LIO (commonly `camera_init`) so new scans appear in a stable world. This process is especially important indoors or in places where GPS signals are unreliable or unavailable.

Another key concept in the rover’s theory is obstacle detection and avoidance. By analyzing the point cloud, the rover can identify which areas are safe to traverse and which contain obstructions. Many autonomous systems also rely on odometry, which is the measurement of movement through wheel rotation or inertial sensors, to estimate changes in position between LiDAR scans. Odometry helps stabilize the mapping process

and improves accuracy when the sensor experiences slight errors or when features temporarily disappear.

The integration of LiDAR with embedded software frameworks such as ROS 2 allows these theoretical principles to be executed in real time. ROS 2 provides tools for visualizing the mapping of LiDAR scans and the tf of LiDAR imu data which all can be done with rviz2. Also, it can manage communication between the rover's sensors and control systems. Together, these theories support the rover's ability to map environments, track its motion, and navigate safely, forming the foundation for more advanced behaviors that will be developed later in the project.

Point-Lio map of woods outside Delta building



CHAPTER 2:

DESIGN AND METHODOLOGY

2.1 Project Statement:

The LiDAR Scout Rover project aims to design and develop a low-cost, modular robotic system capable of performing remote inspection and environmental monitoring in hazardous or inaccessible environments. The primary objective is to enhance situational awareness and reduce risk to human operators by providing real-time mapping, obstacle detection, and environmental sensing through an integrated platform. The system combines a tracked mobile base with onboard computation, LiDAR-based localization, camera streaming, and gas detection, all coordinated through a ROS 2 software framework. By leveraging commercially available components and an open-source architecture, the project emphasizes affordability, scalability, and ease of replication. Ultimately, the LiDAR Scout Rover serves as a practical engineering solution that demonstrates the integration of embedded systems, robotics, and autonomous navigation technologies to address real-world safety and inspection challenges.

*Table 2.1[press **Shift + Enter**]*

Rover Budget (Listed Components)

Component	Model/Example	QTY	Unit Cost (\$)	Subtotal (\$)
Tracked Chassis	Premium Large Metal 4WD Shock Absorption Robot Tank Car Chassis Kit (includes frame, tracks, motors)	1	100.00	100.00
Raspberry Pi 5 Kit (8 GB)	RasTech Pi 5 kit (case, cooler, PSU, SD)	1	64.99	64.99
Motor Drivers	BTS7960 (2-pack) or chassis' integrated controller	1	7.29	7.29
LiDAR	Unitree 4D LiDAR L1 (L1PM 20 m)	1	249.00	249.00
Camera	Arducam IMX219 8MP Autofocus USB Camera Module, 3280(H)x2464(V)@30fps	1	29.99	29.99
Gas Sensors + ADC	MQ kit (MQ-2/7/135...) + MCP3008	1	24.99	24.99
Battery & Power	6S LiPo (22.2 V) + XT90 anti-spark + 5 V/5 A buck + 12V LiPo Battery	1	103.00	103.00
Networking	BrosTrend 650Mbps USB WiFi Adapter (Linux & Pi compatible, long range antenna)	1	23.99	23.99

Mounts & Misc	3D-printed brackets, wiring, fuses, connectors	1	40.00	40.00
Contingency & Incidentals	Extra parts, bolts, tools used	1	80.00	80.00
Total				
Estimated Cost		—	—	768.30

2.2 Project:

The LiDAR Scout Rover project focused on the design, development, and integration of a low-cost, ground-based robotic system capable of remote inspection and environmental monitoring in hazardous or hard-to-reach areas. The system combines a tracked mobile platform with onboard sensing, computation, and communication capabilities to provide real-time situational awareness. Key hardware components include a LiDAR sensor for mapping, a camera for visual feedback, and gas sensors for environmental detection, all coordinated through a Raspberry Pi 5 and an Arduino Mega 2560. The software framework is built on ROS 2, enabling modular communication between perception, localization, and navigation subsystems. The rover is capable of generating maps, detecting obstacles, and navigating semi-autonomously while also supporting manual control through a web-based interface. Emphasis was placed on affordability, modularity, and ease of replication, making the system a practical solution for applications such as search and rescue, confined space inspection, and hazardous site monitoring. Overall, the

project demonstrates the successful integration of embedded systems, robotics software, and mechanical design into a functional and scalable autonomous platform.

2.3 Method:

The LiDAR Scout Rover was developed using a modular systems engineering approach that integrated mechanical, electrical, and software components into a cohesive autonomous platform. The mechanical design utilized a tracked chassis with custom 3D-printed mounts to securely house all subsystems while maintaining stability across uneven terrain. Electrically, the system was powered by dual LiPo batteries with a regulated power distribution network that separated 5V and 12V rails to support the Raspberry Pi 5, motor drivers, sensors, and auxiliary components. An Arduino Mega 2560 was implemented for low-level motor control and sensor interfacing, while a Raspberry Pi 5 served as the primary computing unit running ROS 2 (Jazzy). The software architecture followed a node-based design, enabling communication between perception, localization, and control modules. LiDAR and IMU data were fused using Point-LIO to provide real-time odometry, which was then used by the Nav2 framework for path planning and autonomous navigation. Obstacle detection was performed using the Point Cloud Library (PCL), and a custom proximity guard node enhanced short-range collision avoidance. Additional environmental awareness was achieved through integration of camera and gas sensors, with data accessible through a web-based dashboard for monitoring and teleoperation. This layered approach ensured flexibility, scalability, and reliable performance during real-time operation and testing.

2.4 Deliverables:

The LiDAR Scout Rover project resulted in a complete set of functional and documented deliverables that demonstrate the successful development of the system. The primary deliverable is the fully assembled and operational rover, capable of real-time mapping, obstacle detection, and semi-autonomous navigation using integrated LiDAR, camera, and gas sensing modules. Supporting this hardware is a fully configured ROS 2 software package, including custom nodes for motor control, obstacle avoidance, and sensor integration, as well as implementation of localization and navigation frameworks. Additional deliverables include a web-based dashboard for live monitoring and teleoperation, detailed wiring schematics and system diagrams to support replication and troubleshooting, and CAD models for all custom 3D-printed components. The project also produced comprehensive documentation, including setup instructions, testing procedures, and performance observations, along with a final presentation and demonstration showcasing system capabilities. Together, these deliverables provide a complete, reproducible platform that meets the project's design goals and functional requirements.

2.5 Requirements Gathering – Specifications:

The system requirements for the LiDAR Scout Rover were established through an analysis of operational needs for remote inspection in hazardous and constrained environments, with an emphasis on safety, affordability, and functional reliability. Key functional requirements included the ability to perform real-time mapping using LiDAR data, detect and avoid obstacles, and navigate both autonomously and via manual control. The rover was also required to provide environmental awareness through integrated

camera and gas sensing modules, enabling visual monitoring and detection of hazardous gases. Non-functional requirements focused on maintaining a low-cost design using commercially available components, achieving a stable wireless communication range of approximately 40 meters, and ensuring sufficient battery life for extended operation (targeting multiple hours of runtime). Additional specifications included a compact and durable tracked chassis for multi-terrain mobility, modular software architecture using ROS 2 for scalability and ease of integration, and clear documentation to support system replication and troubleshooting. These requirements guided the overall design process and ensured that the final system met both performance expectations and practical deployment constraints.

2.6 Feature Analysis Based on our User Requirements

The implemented features of the LiDAR Scout Rover were directly derived from the defined user requirements, ensuring that each system's capability addressed a specific operational need. The requirement for real-time situational awareness was fulfilled through the integration of LiDAR-based mapping, camera streaming, and gas sensing, providing both spatial and environmental data to the operator. Autonomous and manual navigation requirements were met using ROS 2 with the Nav2 framework, enabling path planning, obstacle avoidance, and user-controlled teleoperation via a web-based interface. The need for safe operation in unpredictable environments led to the inclusion of obstacle detection using point cloud processing and a custom proximity guard node for short-range collision prevention. Affordability and accessibility requirements were satisfied by selecting cost-effective, commercially available components while maintaining system

performance. Additionally, modularity requirements were addressed through a node-based software architecture, allowing for scalable integration of new sensors and features. Overall, the final system demonstrates strong alignment between user requirements and implemented features, resulting in a practical and adaptable solution for remote inspection tasks.

2.7 System Architecture and Components:

The LiDAR Scout Rover employs a layered system architecture that integrates mechanical, electrical, and software components into a unified and modular platform. At the hardware level, the system consists of a tracked chassis supporting key components including a Raspberry Pi 5 for high-level processing, an Arduino Mega 2560 for low-level motor control, a LiDAR sensor for spatial mapping, a camera for visual feedback, and gas sensors for environmental monitoring. Power is distributed through a dual-voltage system using buck converters to supply regulated 5V and 12V rails to the respective subsystems. At the software level, the architecture is built on ROS 2, where independent nodes handle perception, localization, navigation, and control while communicating through standardized topics. LiDAR and IMU data are processed through a localization module to estimate the rover's pose, which is then used by the navigation stack to generate and execute paths. Additional components include a web-based interface for real-time monitoring and teleoperation, as well as custom nodes for obstacle avoidance and system safety. This modular architecture enables efficient integration, scalability, and maintainability, allowing individual components to be developed, tested, and upgraded independently without affecting the overall system. The rover used a

distributed power layout to support the different voltage requirements of the system. A 22.2 V LiPo battery supplied the main rover system, while buck converters stepped the voltage down for lower-voltage electronics. The Raspberry Pi 5 and USB camera operated from the 5 V supply, while the motor driver and related hardware used the 12 V rail. The LiDAR was powered from its own 12 V battery to reduce interference and improve stability during testing. Separating sensitive sensing equipment from the motor power path helped reduce the chance of voltage drops or electrical noise affecting LiDAR operation.

2.8 Mechanical Integration and 3D-Printed Parts:

Mechanical integration was an important part of the LiDAR Scout Rover design because the sensors, power components, and onboard electronics needed to be mounted securely while keeping the rover balanced and serviceable. The rover used a tracked chassis as the main platform, and additional mounting parts were designed in Fusion 360 to improve component placement and physical protection. These parts included a front plate and skid plate, which helped support the exterior layout of the rover and provided a cleaner mounting surface for hardware.

The 3D-printed parts were printed using PLA filament. PLA was selected because it is inexpensive, easy to print, and strong enough for prototype mounting applications. These printed parts allowed the team to customize the rover layout without needing expensive manufactured brackets. The custom parts also made it easier to adjust component placement during testing as the design changed.

Figure 2.4[press **Shift + Enter**]

3D-Printed Skid Plate in Fusion360



Figure 2.5[press **Shift + Enter**]

3D-Printed Front Plate in Fusion360



Figure 2.6[press **Shift + Enter**]

Point-LIO & NAV 2 System Flowchart

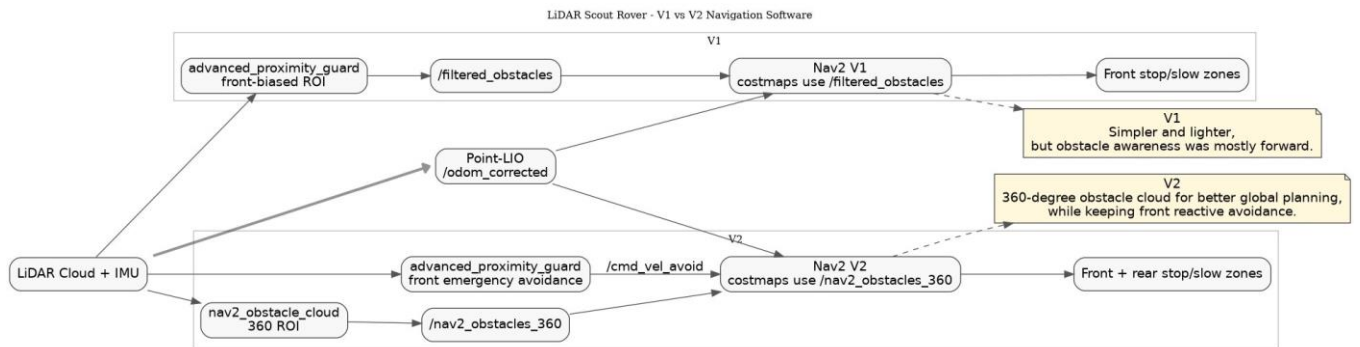
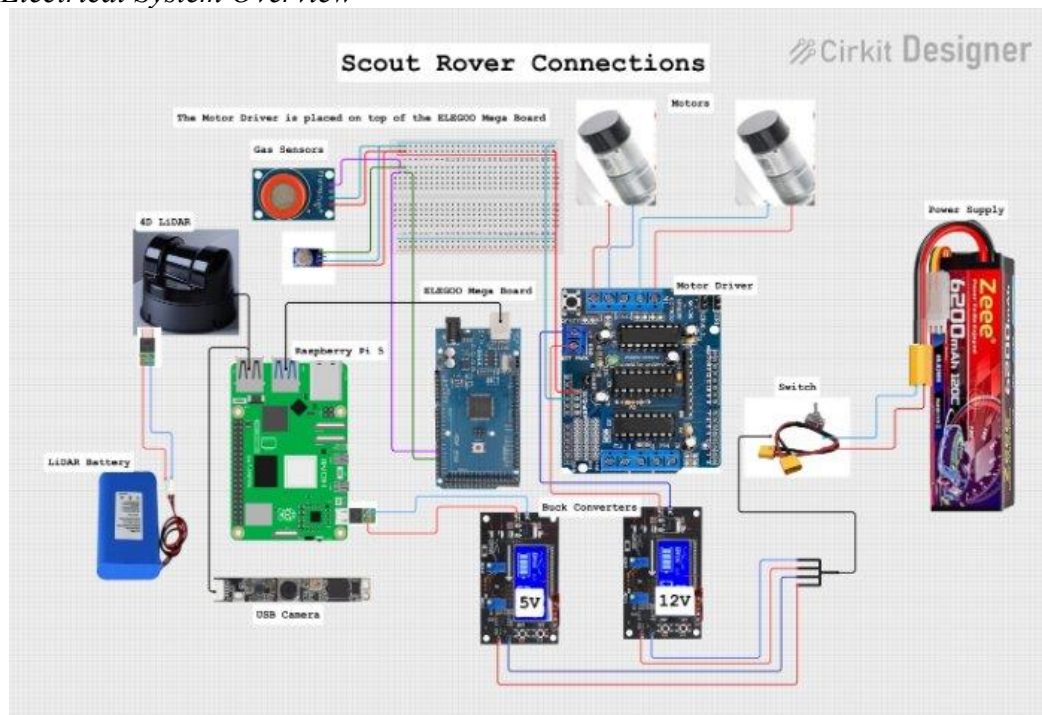


Figure 2.7[press **Shift + Enter**]

Electrical System Overview



CHAPTER 3:

TECHNICAL INVESTIGATIONS & EXPERIMENTS

3.1 Evaluation & Selection of Tools (Software, Hardware, Platform):

Selecting the appropriate tools for the LiDAR Scout Rover required balancing low-level hardware control on the rover with higher compute tasks such as odometry, visualization, and navigation. The Raspberry Pi 5 was selected as the rover-side computer because it can interface directly with the Arduino Mega, Unitree LiDAR driver, gas-sensor bridge, and camera while still running ROS 2 Jazzy on Ubuntu. A separate workstation, used as either an Intel NUC or laptop, was chosen to run Point-LIO, RViz2, and Nav2 because those tools demand more processing power and benefit from a larger display during debugging and demonstrations.

The final platform therefore used a distributed ROS 2 architecture. The Pi 5 published LiDAR point cloud and IMU data, raw gas topics, and camera data, while the workstation consumed those topics over a shared ROS domain and returned motion commands through `/cmd_vel`. A router-based network, with the NUC on Ethernet and the Pi 5 on 5 GHz Wi-Fi, was used together with a shared `ROS_DOMAIN_ID` and CycloneDDS settings so both systems could exchange topics reliably during live testing. The Unitree 4D LiDAR, Arduino Mega 2560, motor driver shield, MQ-series gas sensors, and USB camera were selected because they matched the sensing, mobility, and monitoring requirements of the project and supported the rover's long-term autonomous goals.

3.2 Development of Prototype:

The development of the final prototype moved from basic hardware assembly to a layered software architecture that separated embedded control from higher-level navigation. At the embedded layer, the Arduino Mega was programmed to control the two DC drive motors and sample the MQ-2 and MQ-3 gas sensors. The Raspberry Pi 5 then acted as the bridge between low-level hardware and ROS 2 by running a Python node that subscribed to `/cmd_vel` for motor commands and published raw gas topics after reading the Arduino over serial. The Pi also ran the Unitree LiDAR driver so the point cloud and IMU streams were available to the rest of the system.

At the workstation layer, ROS 2 Jazzy was used to organize mapping, obstacle processing, and navigation into separate nodes and launch files. Point-LIO was integrated to provide LiDAR-inertial odometry, and two navigation workspaces were developed: `ws_lsr_nav2` as the original 180-degree front-focused version and `ws_lsr_nav2_v2` as the improved 360-degree version for broader costmap coverage. Additional nodes were created for obstacle filtering, command multiplexing, gas-data processing, camera viewing, and website communication through rosbridge and a web server. This arrangement allowed the rover

to drive, visualize sensor data, show camera output, read gas values, and execute navigation goals within one connected software stack.

Figure 3.2[press **Shift + Enter**]

NAV 2(Navigation 2) V1

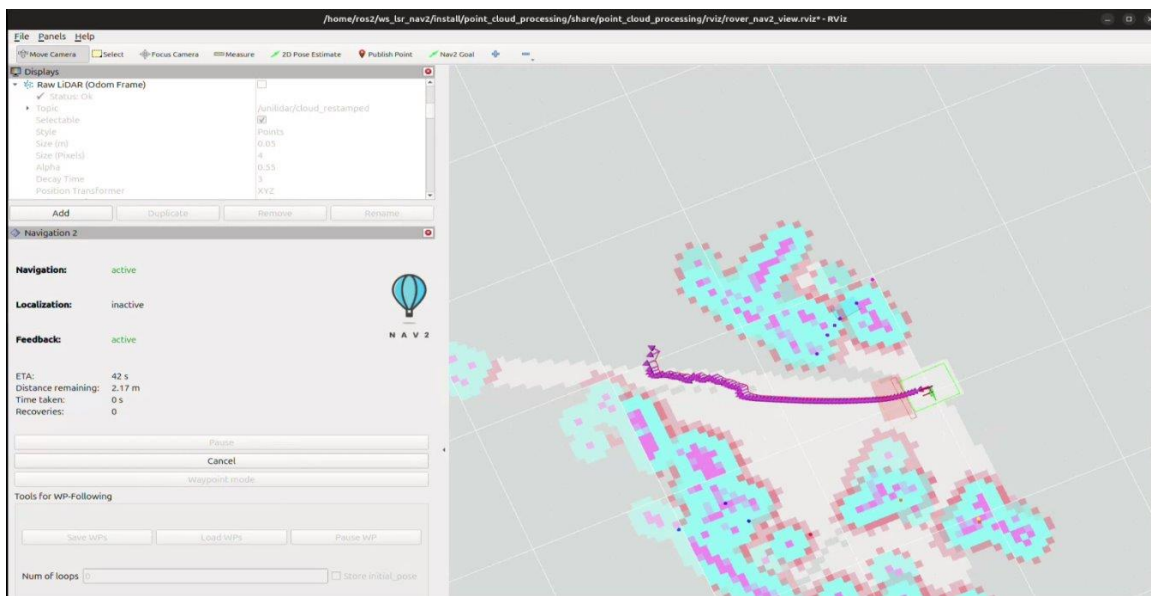
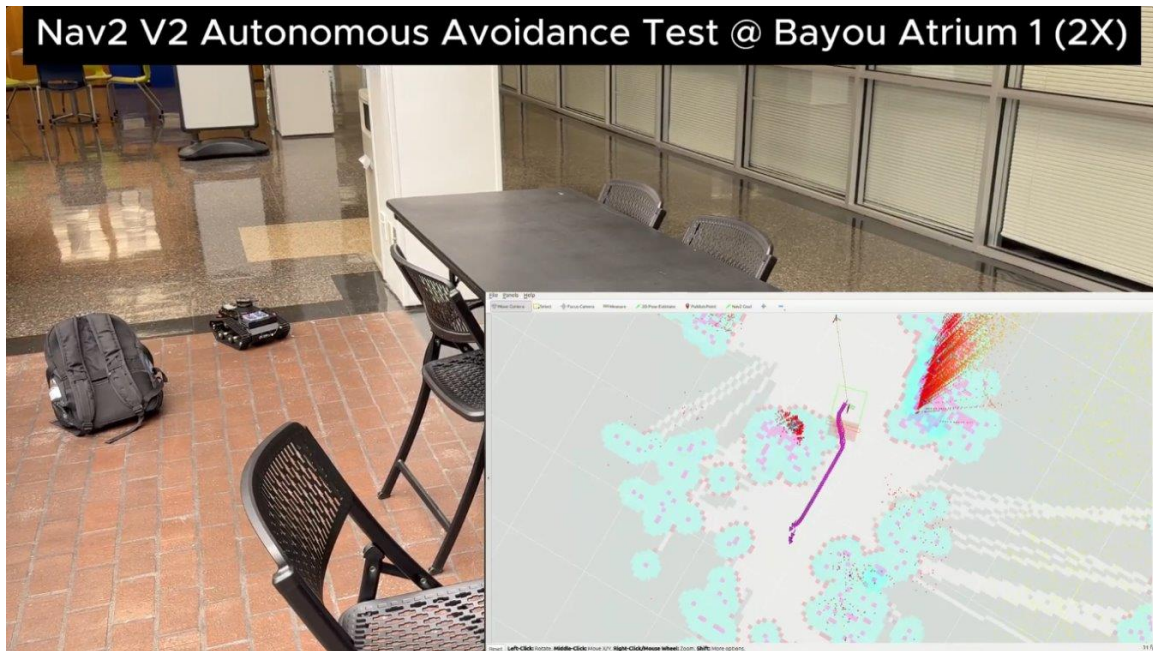


Figure 3.2[press **Shift + Enter**]

NAV 2(Navigation 2) V2 (NAV 2 Goal Start)



3.3 Testing & Debugging:

Testing and debugging centered on synchronization, topic flow, and computational load across the Arduino, Pi 5, and workstation. Early checks verified that all systems shared the same ROS 2 domain and could exchange `cmd_vel`, `gas`, `LiDAR`, `IMU`, and `odometry` topics over the network. The `gas` pipeline was validated by first reading raw values on the Arduino, then relaying them through the Pi bridge, and finally converting them into higher-level ROS 2 topics on the workstation. Similar staged testing was used for camera display and `LiDAR` publishing, so each subsystem could be verified before being combined into the full rover stack.

The most difficult software issue involved timing and computing load during live navigation. When too many heavy nodes and displays were active at once, `RViz2` and the navigation stack began reporting queue overflows, stale obstacle timestamps, and

odometry cutoffs that caused navigation resets. Several improvements were made to address this behavior: the NUC was switched from balanced mode to performance mode, high-load visualization was moved off the robot computer when possible, and the navigation software was separated into a lighter V1 version and a fuller V2 360-degree version. These changes reduced topic dropouts and made live demonstrations more stable.

Debugging also showed that the original front-only obstacle pipeline was too limited when the rover reversed or approached objects from the side. In response, V2 introduced a separate 360-degree obstacle cloud for Nav2 costmaps while keeping the custom front proximity guard as an emergency avoidance layer. The footprint, collision-monitor regions, and near-goal speed settings were then tuned through repeated tests until the rover could approach goals more confidently without stopping too early or becoming blind when backing up. Network troubleshooting was also part of this process, especially when changing between hotspot-based communication and router-based communication, since the CycloneDDS interface settings had to match the active network path.

Through repeated testing and troubleshooting, the system reached a point where it could reliably publish and consume sensor topics across the full software pipeline while also supporting live navigation experiments. Although minor odometry drift and performance-related tuning still remain, the overall ROS 2 system is now stable enough to demonstrate sensing, visualization, and navigation together in one workflow.

3.4 Usability Tests:

Usability testing evaluated how easily an operator could power the rover, connect to the ROS domain, view sensor outputs, and command the rover through the website or RViz2. The Pi 5 was configured to auto-boot the Arduino bridge and LiDAR-side software so the rover could begin publishing core topics with fewer manual steps. On the workstation, the operator could launch Point-LIO, Nav2, and RViz2, confirm live TF and costmaps, and then send goals or teleoperation commands. This workflow made it possible to check the full software stack in a repeatable way before each demonstration.

Additional usability testing focused on the browser interface and visualization pipeline. The website was connected through rosbridge and a local web server so the user could view status information, camera output, and navigation feedback from a browser rather than relying entirely on terminal commands. These tests showed that the overall system became easier to operate once the software was separated into clear rover-side and workstation-side responsibilities. The main usability improvements still needed are further startup automation, cleaner handling of network-interface settings, and continued simplification of launch procedures for demonstrations.

3.5 Results / Findings:

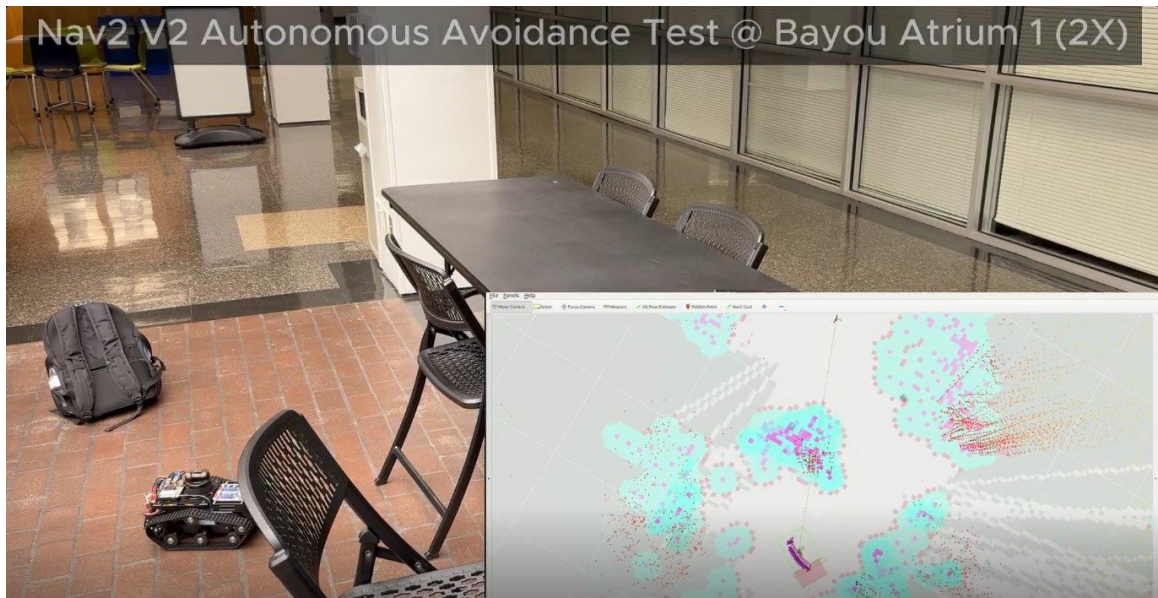
The final software system achieved the main goals set for the rover's functional behavior. The rover was able to drive in commanded directions, generate LiDAR-based maps with Point-LIO, display live camera data, publish gas sensor topics, and navigate with obstacle avoidance using Nav2. The Pi 5 successfully served as the hardware interface for the

Arduino, LiDAR, and camera, while the workstation handled the heavier odometry and navigation tasks. This distributed design allowed the project to move beyond basic sensor visualization and into real navigation experiments.

Comparative testing between the two navigation workspaces also produced a clear finding. The original V1 workspace, which emphasized a forward field of view, was simpler and sometimes appeared steadier, but it was limited during reversing and side-approach cases. The V2 workspace added 360-degree obstacle processing and produced better costmap coverage for global planning, making it the stronger foundation for continued development. Although minor TF drift and performance-related tuning still remain, the rover now demonstrates mapping, obstacle awareness, gas monitoring, camera display, and browser-based interaction in a single ROS 2 system.

*Figure 3.5[press **Shift + Enter**]*

NAV 2(Navigation 2) V2 (NAV 2 Goal Reached)



CHAPTER 4:

PROJECT MANAGEMENT

4.1 Project Management:

The management of the LiDAR Scout Rover project focused on coordinating hardware integration with a distributed software workflow across the Arduino, Pi 5, and workstation. Because the rover combined mechanical assembly, power regulation, embedded control, perception, navigation, and web development, tasks were divided into focused roles and then integrated through regular testing sessions. This approach allowed hardware work, ROS 2 node development, and interface development to progress in parallel while still meeting shared milestones.

Regular checkpoints were used to track issues such as serial communication, ROS topic discovery, timing mismatches, and navigation stability. As the project matured, more time was allocated to Point-LIO configuration, obstacle-processing nodes, gas and camera integration, and refinement of the two Nav2 workspaces. The change from a simpler front-focused navigation stack to a stronger 360-degree V2 stack required additional iteration, but documenting each test made it easier to identify which settings improved performance and which changes introduced regressions.

As development continues, project management will focus on tightening system reliability rather than assembling core functionality from scratch. Future schedules should

dedicate time to odometry tuning, startup automation, broader field testing, and continued refinement of the website and sensor dashboards. This structured workflow has already helped turn the rover into an integrated sensing and navigation platform, and it will remain important as the team pushes toward more reliable autonomous operation. [OBJ]

CHAPTER 5:

CONCLUSION AND FUTURE WORK

5.1 Conclusion and Future Work:

The final version of the LiDAR Scout Rover established a complete multi-computer software architecture that connects embedded motor control, gas sensing, LiDAR perception, camera streaming, odometry, and navigation into one ROS 2 workflow. The Arduino Mega handled low-level motor and gas acquisition, the Pi 5 bridged hardware data into ROS 2 and published LiDAR and camera topics, and the workstation ran Point-LIO, Nav2, RViz2, and the higher-level visualization tools. As a result, the rover progressed from a basic sensing prototype to a system capable of mapping, obstacle avoidance, gas reporting, and semi-autonomous operation.

Future work will focus on improving stability rather than building the software stack from the beginning. The most important next steps are refining TF drift, tuning odometry and Nav2 behavior for tighter final positioning, calibrating gas thresholds, and expanding website integration so more rover status can be monitored from the browser. The project also demonstrated the importance of mechanical integration, power routing, and custom 3D-printed parts in creating a rover platform that was stable, serviceable, and practical for

repeated testing. Additional improvements may include further startup automation, more polished RViz and URDF visualization, and continued testing of the 360-degree navigation pipeline in larger environments. With these refinements, the project is positioned to become a more reliable field-ready inspection rover.

BIBLIOGRAPHY

[1] Amazon.com: Swaytail Premium Large Metal 4WD shock absorption robot tank car chassis kit for Arduino/Raspberry Pie/MicroBit with 33 DC motor, shock absorbing tracked crawler with damping system DIY Maker Model : Toys & Games, <https://www.amazon.com/Premium-Absorption-Raspberry-MicroBit-Absorbing/dp/B09CYDB8H4> (accessed Nov. 19, 2025)

[2] Amazon.com: For Raspberry Pi 5 8GB starter kit includes with Raspberry Pi 5 8GB, PI5 case with active cooler, 27W USB type-C power supply, 64GB SD card and TF Card Reader, HDMI cable, ethernet cable (8 items) : Electronics, <https://www.amazon.com/Raspberry-Starter-Active-Cooler-Ethernet/dp/B0CXXFFYNB> (accessed Nov. 19, 2025).

[3] Walmart.com, L293D Motor Drive Shield Expansion Board For Arduino Uno Mega2560 & Duemilanove, Envistia Mall, <https://www.walmart.com/ip/L293D-Motor-Drive-Shield-Expansion-Board-For-Arduino-Uno-MEGA2560-Duemilanove/400667780?wmlspartner=wlpa&selectedSellerId=101028229> (accessed Apr. 12, 2025)

[4] "4D Lidar L1 application Scenarios_4D LIDAR L1 efficacy: Unitree Robotics," 4D Lidar L1 Application Scenarios_4D Lidar L1 Efficacy | Unitree Robotics, <https://www.unitree.com/LiDAR/> (accessed Nov. 19, 2025).

[5] Amazon.com: Arducam IMX219 8MP Autofocus USB Camera Module, 3280(H)x2464(V)@30fps with Single Microphone for Linux, Windows, and Mac OS, <https://a.co/d/0fThY8IZ> (accessed Apr. 4, 2026).

[6] Amazon.com: CJRSLRB® Gas Detection Module MQ-2 MQ-3 MQ-4 MQ-5 MQ-6 MQ-7 MQ-8 MQ-9 MQ-135 sensor module each of them 1pcs total 9pcs sensor kit : Tools & Home Improvement, <https://www.amazon.com/CJRSLRB-Detection-Module-MQ-135-Sensor/dp/B016KABTDK> (accessed Nov. 19, 2025).

[7] Mavdrones, "Tattu 2200 mah lipo battery with XT90-S Anti-Spark connector," Mavdrones, <https://www.mavdrones.com/product/tattu-22-2v-25c-6s-2200-mah-lipo-battery-with-xt90-s-anti-spark-connector/> (accessed Nov. 19, 2025).

[8] Amazon.com: BrosTrend 650mbps linux compatible WIFI adapter supports Kali Linux, ubuntu, Mint, debian, kubuntu, zorin, pureos,

Raspberry Pi 2+, windows, dual band USB Wireless Adapter W/ long range WIFI antenna : Electronics,

<https://www.amazon.com/BrosTrend-600Mbps-Wireless-Internet-AC2/dp/B01GC8XH0S> (accessed Nov. 19, 2025).

[9] E. Ackerman, "Boston Dynamics' Spot Robot Dog now available for \$74,500," IEEE Spectrum, <https://spectrum.ieee.org/boston-dynamics-spot-robot-dog-now-available> (accessed Nov. 19, 2025).

[10] Base Year:2024|, "Autonomous Mobile Robots Market Size (\$13.7 billion) 2030," STRATEGIC MARKET RESEARCH, <https://www.strategicmarketresearch.com/market-report/autonomous-mobile-robots-market> (accessed Nov. 19, 2025).

[11] D. He, W. Xu, N. Chen, F. Kong, C. Yuan, and F. Zhang, "Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry," *Advanced intelligent systems*, vol. 5, no. 7, Apr. 2023, doi: <https://doi.org/10.1002/aisy.202200459>.

[12] Adam Zewe | MIT News, "Engineers enable a drone to determine its position in the dark and indoors," MIT News | Massachusetts Institute of

Technology, <https://news.mit.edu/2025/engineers-enable-drone-determine-its-position-dark-and-indoors-0213> (accessed Nov. 19, 2025).

[13] "What is a Lithium Polymer Battery? | LiPo," *Ossila*, 2021.

<https://www.ossila.com/pages/lithium-polymer-battery>

[14] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022, doi:

<https://doi.org/10.1126/scirobotics.abm6074>.

[15] "ROS 2 Documentation: Jazzy documentation," ROS 2 Documentation,

<https://docs.ros.org/en/jazzy/index.html> (accessed Sept. 2025).

[16] "Nav2," Navigation2 Documentation, <https://docs.nav2.org/> (accessed Jan. 2026).

- [17] "Costmap 2D," Navigation2 Documentation,
<https://docs.nav2.org/configuration/packages/configuring-costmaps.html>
(accessed Jan. 2026).
- [18] "RViz," ROS 2 Documentation: Jazzy documentation,
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/RViz/RViz-Main.html>
(accessed Sept. 2025).
- [19] Point Cloud Library (PCL), PointClouds.org, <https://pointclouds.org/>
(accessed Apr. 1, 2026).
- [20] "ROS 2 Jazzy Jalisco Released!," Open Robotics Discourse,
<https://discourse.openrobotics.org/t/ros-2-jazzy-jalisco-released/37862>
(accessed Sept. 2025).